#1 데이터 타입 기초 실습

- **목표**: SystemVerilog의 기본 데이터 타입(bit, logic, int, byte, enum)을 직접 선언하고, 비트 기반 값과 정수 기반 값을 서로 변환해보며 데이터 타입에 대한 감각을 익힌다. 아래 템플릿 코드의 빈칸을 채워 완성하라.
- **요구 기능**
  - bit, logic, byte, int 타입 변수를 선언
  - 4비트 이진수 값을 bit[3:0] 타입 변수에 저장
  - 동일한 값을 int로 변환해서 출력
  - 연산 상태를 enum으로 선언하고 특정 상태일 때 메시지를 출력

```
module datatype_practice;

  // TODO 1: 기본 타입 선언
  bit     b1;
  logic   l1;
  byte    by1;
  int     num;

  // TODO 2: 4비트 이진수 변수를 선언하고 값 대입
  bit [3:0] nibble = 4'b____;

  // TODO 3: 동일한 값을 정수형 num에 변환하여 저장
  initial begin
    num = ____;   // nibble 값을 정수로 변환

    $display("Nibble(binary) = %b, Nibble(int) = %0d", nibble, num);
  end

  // TODO 4: 연산 상태 enum 정의
  typedef enum { IDLE, RUN, DONE } state_t;

  // 상태 변수 선언
  state_t st = _____;

  initial begin
    if (st == RUN) begin
      $display("State = RUN");
    end else begin
      $display("State != RUN");
    end
  end

Endmodule
```

#1 데이터 타입 기초 실습(answer)

```
module datatype_practice;

  // 기본 타입 선언
  bit     b1;
  logic   l1;
  byte    by1;
  int     num;

  // 4비트 이진수 값
  bit [3:0] nibble = 4'b1010;
 initial begin
    // bit[3:0] 값을 int로 자동 변환
    num = nibble;

    $display("Nibble = %b, as int = %0d",
nibble, num);
  end
```

```
  typedef enum { IDLE, RUN, STOP } state_t;

  // 상태 초기값
  state_t st = IDLE;

  initial begin
    if (st == RUN)
      $display("State is RUN");
    else
      $display("State is not RUN");
  end

endmodule
```

#2 디버깅 실습 — 오류 찾고 수정하기

- **목표**
  기본 타입 및 enum 관련 오류를 스스로 해결해본다.
- **문제 설명**
  아래 코드는 타입 선언, 문자열, 배열, enum 사용에 오류 또는 미묘한 문제를 포함하고 있다.
  코드가 정상 동작하도록 모든 오류를 수정하라.

```
module debug_dt;

  bit [7:0] data;
  int value;

  // 오류 포함된 enum
  typedef enum { ADD=1, SUB, MUL=4, div } op_t;

  op_t current = Div;    // 대소문자 오류 포함

  string s = 8'h41;      // 문자열 초기화 오류

 initial begin
    data = 12.8;         // 실수 대입 오류

    if (current = ADD)   // '=' vs '==' 오류
      $display("ADD!");

    // 문자열 출력 오류
    $display("s=%b", s);

    value = data;  // 의도치 않은 타입 변환

    $display("value=%d", value);
  end

endmodule
```

#2 디버깅 실습 — 오류 찾고 수정하기(answer)

```
module debug_dt;

  bit [7:0] data;
  int value;

  // 올바른 enum 정의
  typedef enum { ADD=1, SUB, MUL=4, DIV } op_t;

  // 올바른 enum 값 사용
  op_t current = DIV;

  // 문자열 초기화 (ASCII 'A' = 8'h41)
  string s = "A";

  initial begin
    // 정수값 대입 (실수 금지)
    data = 8'd12;

    // 비교 연산자 == 사용
    if (current == ADD)
      $display("ADD!");

    // 문자열 출력은 %s 사용
    $display("s=%s", s);

    // bit → int 정상 변환
    value = data;
    $display("value=%d", value);
  end

endmodule
```

#3 데이터 타입 기초 실습

- **목표: 문자열과 고정 배열의 차이, 인덱스 접근 등을 연습한다.**
- **문제: 아래 요구사항을 만족하는 프로그램을 작성하라.**
- **요구 기능**
  - **char 타입 배열 char msg[6] 를 선언하고**
  - **“HELLO”라는 문자를 순서대로 저장한다.**
  - **string 타입 변수 s 에 "WORLD"를 저장한다.**
  - **msg와 s의 3번째 문자(index 2)를 출력한다.**
  - **msg 배열 내용 전체를 문자열로 묶어 출력한다.**

- TIP!
  - initial 블록만 자유롭게 작성.
  - msg[0] = "H";
  - 문자열 결합은 string cast 필요.

#3 데이터 타입 기초 실습(answer)

```
module string_array_practice;

  char msg[6];    // H E L L O + NULL
  string s;

  initial begin
    // msg 배열에 "HELLO" 저장
    msg[0] = "H";
    msg[1] = "E";
    msg[2] = "L";
    msg[3] = "L";
    msg[4] = "O";
    msg[5] = 0;   // NULL termination

    // 문자열 변수
    s = "WORLD";

    // 3번째 문자 출력
    $display("msg[2] = %s", msg[2]);
    $display("s[2]   = %s", s[2]);

    // msg 전체 출력 - 캐스팅 필요
    string msg_str;
    msg_str = { msg[0], msg[1], msg[2], msg[3],
msg[4] };
    $display("msg string = %s", msg_str);
  end
endmodule
```

#4 Struct + Enum 활용 기초

- 목표: 구조체(struct)와 enum을 동시에 사용하는 연습을 한다.

- 문제: 아래 템플릿의 TODO를 채워라.

```
module packet_info;

  // Packet type 정의
  typedef enum { READ, WRITE, ERROR } pkt_t;

  // Packet 구조체 선언
  typedef struct {
    pkt_t   type;
    int     addr;
    int     data;
  } packet_t;

  // TODO: packet_t 변수를 선언하고 직접 필드 초기화
  packet_t p1;
```

```
 initial begin
    // TODO: p1.type  = WRITE;
    // TODO: p1.addr  = 32'h1000_0000;
    // TODO: p1.data  = 1234;

    // TODO: $display로 p1 내용을 출력하라
  end

endmodule
```

#4 Struct + Enum 활용 기초(answer)

```
module packet_info;

  typedef enum { READ, WRITE, ERROR } pkt_t;

  typedef struct {
    pkt_t   type;
    int     addr;
    int     data;
  } packet_t;

  packet_t p1;

  initial begin
    p1.type = WRITE;
    p1.addr = 32'h1000_0000;
    p1.data = 1234;

    $display("Packet Info: type=%0d addr=%h data=%0d",
             p1.type, p1.addr, p1.data);
  end

endmodule
```

#5 Mini Register File Simulator

- **목표: 데이터 타입, 배열, struct, enum, task를 종합적으로 연습한다.**
- **문제: 아래 요구사항을 모두 만족하는 register file simulator를 작성하라.**
- **요구 기능**
  1. 레지스터 타입 정의
     - 32비트 정수 저장
     - 주소(addr), 데이터(data)
  2. 레지스터 배열 regfile[4] 생성
     - 주소는 0,1,2,3으로 초기화
     - data는 모두 0으로 초기화
  3. enum으로 operation 정의
     - RF_WRITE
     - RF_READ
  4. task reg_access() 구현
     인자:
     - input op_t op
     - input int addr
     - input int wdata
     - output int rdata
  5. 동작:
     1. op == RF_WRITE → 해당 addr에 wdata 저장
     2. op == RF_READ → 해당 addr의 값을 rdata에 반환
     3. 범위 밖 주소면 rdata = -1로 설정
  6. initial 블록에서 다음 시나리오를 수행
     1. 주소 2에 55 쓰기
     2. 주소 2 읽기
     3. 주소 1 읽기(초기값 0)
     4. 주소 10 읽기(범위 밖)
  7. 실행 결과를 $display로 출력
- 출력 예

Write: addr=2 data=55 Read: addr=2 data=55 Read: addr=1 data=0 Read: addr=10 data=-1 (invalid)

#5 Mini Register File Simulator(answer)

```
module regfile_sim;

  typedef struct {
    int addr;
    int data;
  } reg_t;

  // 레지스터 파일 4개
  reg_t regfile[4];

  // Operation enum
  typedef enum { RF_WRITE, RF_READ } op_t;

  // 레지스터 접근 task
  task reg_access(
      input  op_t op,
      input  int addr,
      input  int wdata,
      output int rdata
  );
    if (addr < 0 || addr > 3) begin
      rdata = -1;        // invalid address
    end
    else begin
      case (op)
        RF_WRITE: regfile[addr].data = wdata;
        RF_READ:  rdata = regfile[addr].data;
      endcase
    end
  endtask

  initial begin
    int rd;

    // 초기화
    for (int i = 0; i < 4; i++) begin
      regfile[i].addr = i;
      regfile[i].data = 0;
    end

    // 1) 주소 2에 55 쓰기
    reg_access(RF_WRITE, 2, 55, rd);
    $display("Write: addr=2 data=55");

    // 2) 주소 2 읽기
    reg_access(RF_READ, 2, 0, rd);
    $display("Read:  addr=2 data=%0d", rd);

    // 3) 주소 1 읽기
    reg_access(RF_READ, 1, 0, rd);
    $display("Read:  addr=1 data=%0d", rd);

    // 4) 주소 10 읽기 (invalid)
    reg_access(RF_READ, 10, 0, rd);
    $display("Read:  addr=10 data=%0d (invalid)", rd);
  end

endmodule
```